

Time and Space Complexity

Where We Left Off

We can write loops over arrays of any size

Some programs finish instantly

Some take minutes on the same data

How do we **compare** two programs
before running either one?

Two Resources

Time how many operations must run

Space how much memory is needed

Both grow with the size of the input

On a microcontroller, **space** is often the binding limit

Why Not Just Time It?

Run it and measure - but the answer depends on:

the **machine**

the **input** used

the **compiler flags**

what else is running

-O0 versus -O2 alone can change it 10 times

We want a property of the **algorithm**

Count Operations Instead

Count the **basic steps** the algorithm takes

Express the count as a function of the **input size**, called **n**

$T(n)$ = the number of steps

A machine-independent measure

What Is n ?

It depends on the problem:

the number of **array elements**

the **length of a string**

the number of **samples** in the log

the number of **pixels** in an image

Always say what n means before you analyze

Counting Steps

```
float sum = 0.0f ;           1
for ( int i = 0 ; i < n ; i++ ) 1 + (n+1) + n
    sum += psi[i] ;           n
return sum / n ;              2
```

$$T(n) = 3n + 5$$

The Constants Do Not Matter

Your count: $T(n) = 3n + 5$

Counting differently: $T(n) = 5n + 2$

A faster machine: $T(n) = 1.5n + 2$

All three are **straight lines**

Machines and compilers change the **constants**

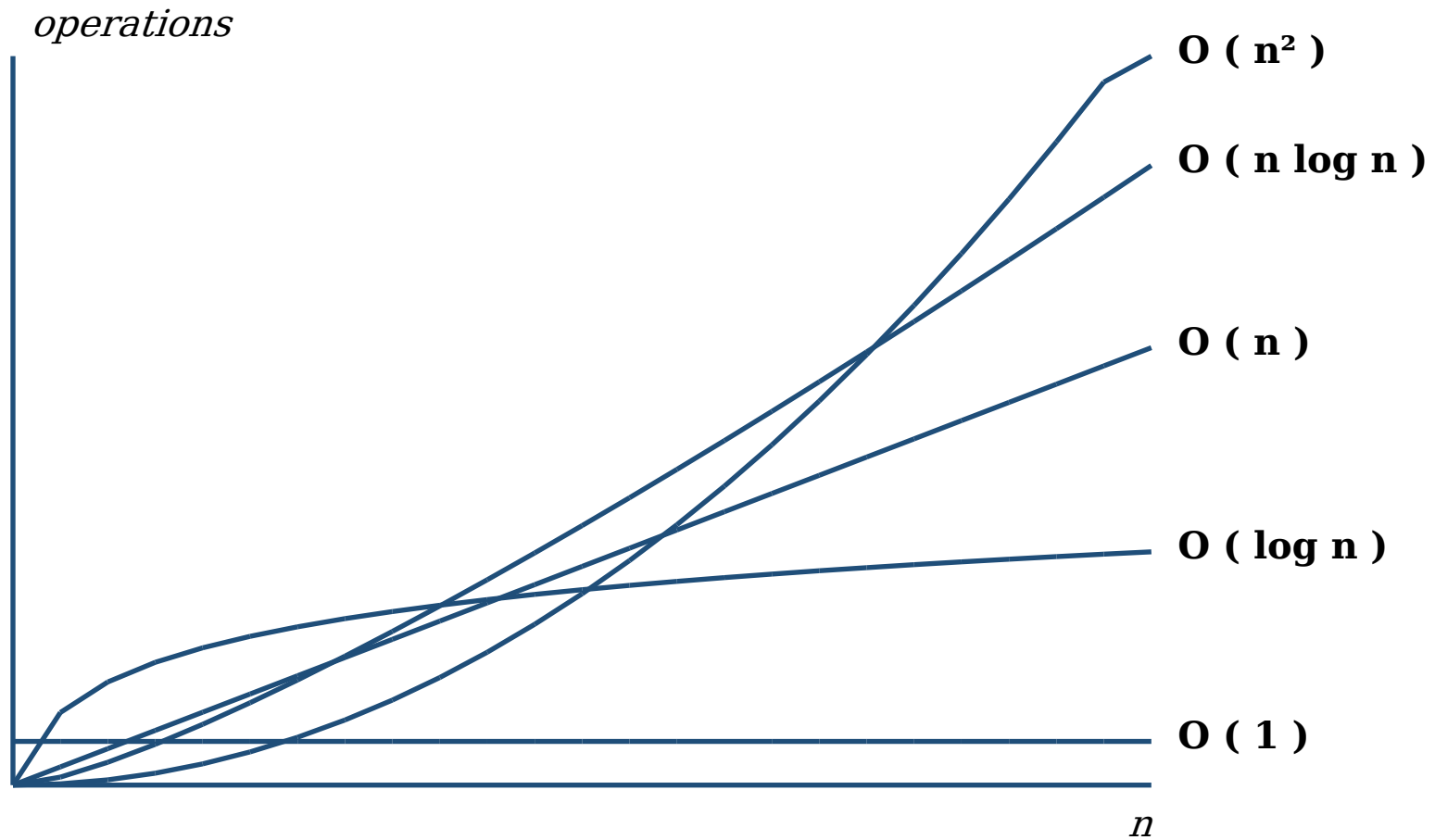
The algorithm sets the **shape**

Growth Is What Matters

n	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$
10	1	3	10	33	100
100	1	7	100	664	10,000
1,000	1	10	1,000	9,966	1,000,000
1,000,000	1	20	1,000,000	20 million	10^{12}

Approximate number of basic operations

The Same Picture



Put It in Time

At one **billion** operations per second,
with $n = 1,000,000$:

$O(n)$	about 1 millisecond
$O(n \log n)$	about 20 milliseconds
$O(n^2)$	about 17 minutes

Same computer. Same data. Different algorithm.

Big-O: The Idea

A way to say:

"f grows no faster than g"

Ignore constant factors

Ignore small n

Keep only the **shape** of the growth

Big-O: The Definition

$$f(n) = O(g(n))$$

if there are constants $c > 0$ and $n_0 \geq 0$
such that

$$0 \leq f(n) \leq c \cdot g(n) \quad \text{for all } n \geq n_0$$

g bounds f from **above**, eventually

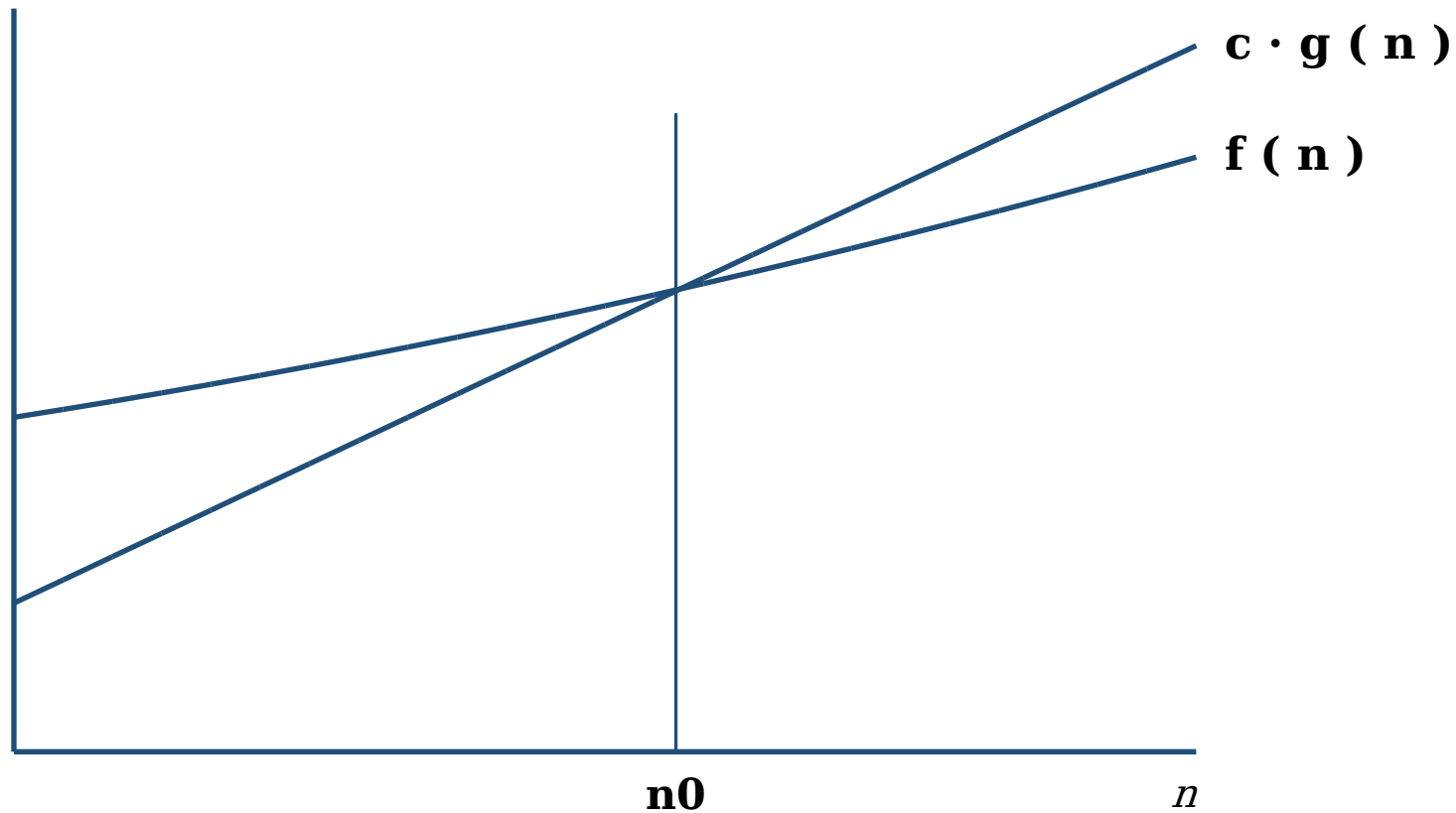
Reading the Definition

c lets us scale g up
 constant factors do not matter

n0 lets us ignore small inputs
 only **large** n matters

$n \geq n0$ the bound never fails again

Big-O in a Picture



For every $n \geq n_0$: $f(n) \leq c \cdot g(n)$

Proving $3n + 2 = O(n)$

Choose $c = 4$ and $n_0 = 2$

Claim: $3n + 2 \leq 4n$ for all $n \geq 2$

Check: $3n + 2 \leq 4n$
 $2 \leq n$ true for $n \geq 2$

So $3n + 2 = O(n)$

Proving $5n^2 + 3n + 1 = O(n^2)$

For $n \geq 1$:

$$3n \leq 3n^2 \quad \text{and} \quad 1 \leq n^2$$

$$5n^2 + 3n + 1 \leq 5n^2 + 3n^2 + n^2 = 9n^2$$

So with $c = 9$ and $n_0 = 1$:

$$5n^2 + 3n + 1 = O(n^2)$$

Rules of Thumb

Drop **constant factors** $5n \rightarrow O(n)$

Drop **lower-order terms** $n^2 + n \rightarrow O(n^2)$

Sequential code **adds** $O(n) + O(n) = O(n)$

Nested loops **multiply** $O(n) \times O(n) = O(n^2)$

Big-O Is Only an Upper Bound

$n = O(n^2)$ **true**

$n = O(n^5)$ **true**

Both are correct and both are useless

Always state the **tightest** bound you can
prove: $n = O(n)$

IDE

Integrated Development Environment (IDE)

All the software tools needed to create a program

IDE

Integrated Development Environment (IDE)

All the software tools needed to create a program

Omega: The Lower Bound

$$f(n) = \Omega(g(n))$$

if there are constants $c > 0$ and $n_0 \geq 0$
such that

$$0 \leq c \cdot g(n) \leq f(n) \quad \text{for all } n \geq n_0$$

"f grows **at least** as fast as g"

Theta: The Tight Bound

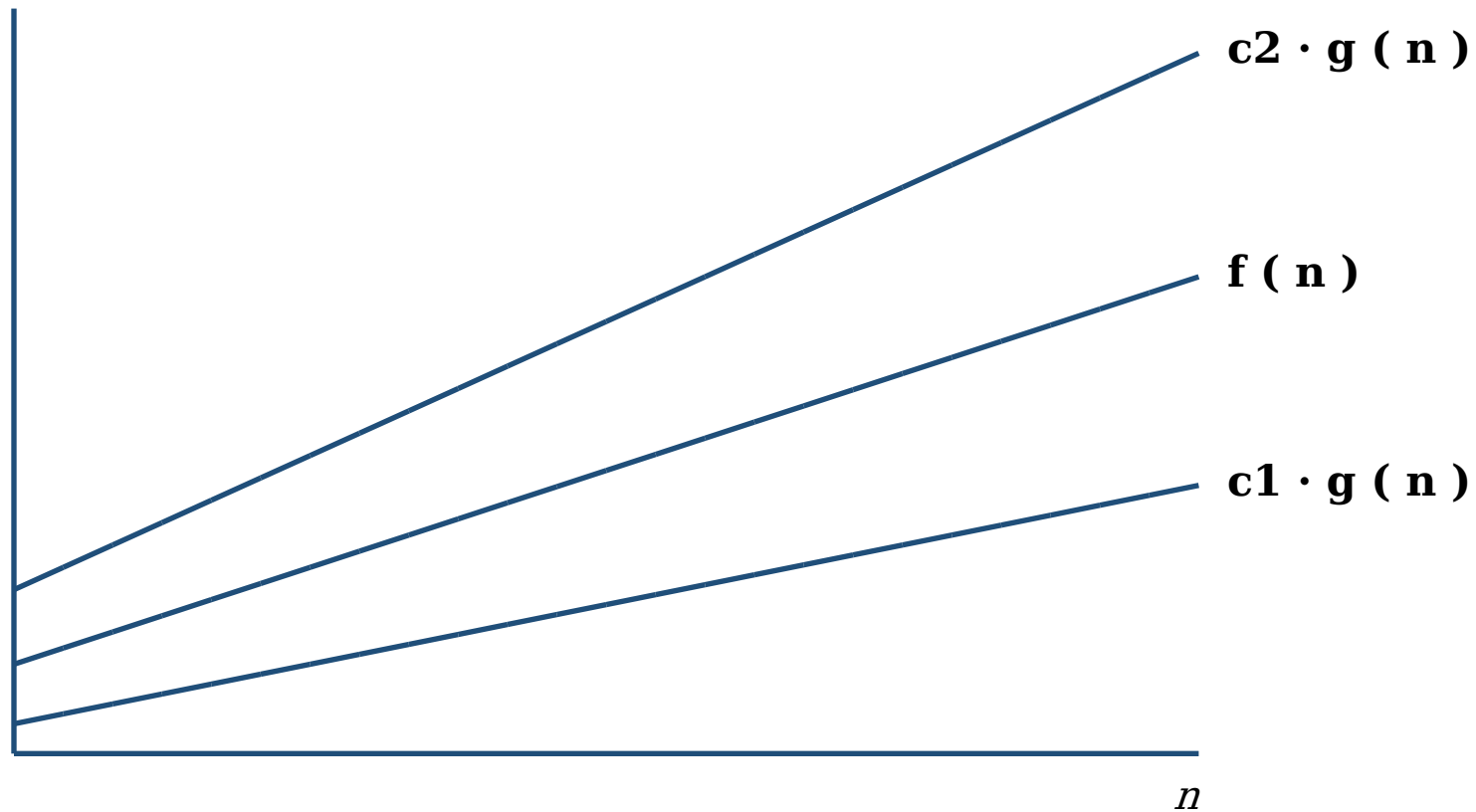
$$f(n) = \Theta(g(n))$$

if there are constants $c_1, c_2 > 0$ and n_0
such that

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \text{for } n \geq n_0$$

$f = \Theta(g)$ means **O and Ω**

The Three Together



O : ceiling

Ω : floor

Θ : both

All Three for $3n + 2$

$$O(n) : c = 4, n_0 = 2 \quad 3n + 2 \leq 4n$$

$$\Omega(n) : c = 3, n_0 = 1 \quad 3n \leq 3n + 2$$

$\Theta(n)$: both hold, so the bound is **tight**

Why We Concentrate on Big-O

We usually want a **guarantee**:
"it will take no longer than this"

Big-O states the **worst case** cleanly

In practice, people say O when they
have proved Θ - know the difference,
but expect the shorthand

The Common Classes

$O(1)$	constant
$O(\log n)$	logarithmic
$O(n)$	linear
$O(n \log n)$	linearithmic
$O(n^2)$	quadratic
$O(2^n)$	exponential
$O(n!)$	factorial

Anything past n^2 needs a very small n

$O(1)$: Constant Time

The work does **not** depend on n

C: `float p = psi[i] ;`

C++: `float p = v[i] ;`

Rust: `let p = v[i] ;`

Also: struct member access, arithmetic,
push to the end of a vector (amortized)

$O(n)$: Linear Time

Touch every element **once**

C: `for ( i = 0 ; i < n ; i++ ) sum += psi[i] ;`

C++: `for ( float p : v ) sum += p ;`

Rust: `for p in v.iter ( ) { sum += *p ; }`

Double `n`, double the work

$O(n^2)$: Quadratic Time

A loop inside a loop, both over n

C:

```
for ( i = 0 ; i < n ; i++ )  
    for ( j = 0 ; j < n ; j++ )  
        compare ( a[i], a[j] ) ;
```

Rust:

```
for i in 0..n { for j in 0..n { ... } }
```

Double n , **four times** the work

Triangular Loops Are Still n^2

```
for ( i = 0 ; i < n ; i++ )  
    for ( j = i + 1 ; j < n ; j++ )    ...
```

Iterations: $1 + 2 + \dots + (n-1) = n(n-1) / 2$

$$= n^2/2 - n/2 = O(n^2)$$

Half the work is still quadratic growth

$O(\log n)$: Binary Search

The array must be **sorted**

Look at the middle element:

equal found

too big search the **left** half

too small search the **right** half

Each step throws away **half** the data

Binary Search in C

```
int find ( const int a[], int n, int key ) {  
    int lo = 0, hi = n - 1 ;  
    while ( lo <= hi ) {  
        int mid = lo + ( hi - lo ) / 2 ;  
        if ( a[mid] == key )    return mid ;  
        if ( a[mid] < key )    lo = mid + 1 ;  
        else                   hi = mid - 1 ;  
    }  
    return -1 ;  
}
```

Binary Search in C++ and Rust

C++:

```
#include <algorithm>
bool found = std::binary_search ( v.begin ( ), v.end ( ),
key ) ;
auto it = std::lower_bound ( v.begin ( ), v.end ( ), key ) ;
```

Rust:

```
match v.binary_search ( &key ) {
    Ok ( i ) => /* found at i */,
    Err ( i ) => /* not found; insert at i */,
}
```

Why Halving Gives $\log n$

Step	Elements left
0	1,000
1	500
2	250
3	125
4	63
5	32
6	16
7	8
8	4
9	2
10	1

$\log_2 (1000) \approx 10$. A million elements need only 20 steps.

$O(n \log n)$: Good Sorting

C: `qsort ( a, n, sizeof a[0], cmp ) ;`

C++: `std::sort ( v.begin ( ), v.end ( ) ) ;`

Rust: `v.sort ( ) ;` stable
`v.sort_unstable ( ) ;` usually faster

No comparison sort can beat $\Omega(n \log n)$

$O(2^n)$: Exponential

```
unsigned long fib ( unsigned n ) {  
    if ( n < 2 ) return n ;  
    return fib ( n - 1 ) + fib ( n - 2 ) ;  
}
```

Every call makes **two** more calls

fib (50) makes about 10^{10} calls - hours

How to Analyze Code

1. Say what **n** is
2. Count the **iterations** of each loop
3. Multiply for **nested** loops
4. Add for code in **sequence**
5. Keep the **largest** term

Sequence Adds

Pass 1: sum the array $O(n)$

Pass 2: find the maximum $O(n)$

Total: $O(n) + O(n) = O(2n) = \mathbf{O(n)}$

Two passes over the data are still linear

Loops That Ignore n

```
for ( int k = 0 ; k < 8 ; k++ )      O ( 1 )  
    total += counts >> k & 1 ;
```

8 is a **constant**, not n

A loop over the bits of a fixed-size integer
is constant time

Function Calls Count Too

The cost of a call is the cost of its **body**

`sum += adc_to_psi ( counts[i] ) ;` $O(1)$ each

`process ( array, n ) ;` $O(n)$ each

A call inside a loop **multiplies**

The strlen Trap

```
for ( int i = 0 ; i < strlen ( s ) ; i++ )      O ( n2 )  
    s[i] = toupper ( s[i] ) ;
```

strlen walks the whole string **every** pass

```
size_t len = strlen ( s ) ;  
for ( int i = 0 ; i < len ; i++ )      O ( n )
```

Best, Average, Worst Case

Linear search for a key in n elements:

best	first element	$O(1)$
average	halfway	$O(n)$
worst	last or missing	$O(n)$

We normally quote the **worst** case

Space Complexity

How much **extra** memory the algorithm needs,
as a function of n

Count what you allocate **beyond the input**

Stack frames count

Heap allocations count

O (1) Space

```
float sum = 0.0f ;  
for ( int i = 0 ; i < n ; i++ )  
    sum += psi[i] ;
```

Two extra variables, whatever n is

Also called working **in place**

O (n) Space

Make a copy of the data:

```
let doubled : Vec<f32> = v.iter ( )  
    .map ( | p | p * 2.0 ).collect ( ) ;
```

Build a new array as long as the input

Or recurse n deep - each call is a **frame**

Recursion Costs Space

factorial (5) 5 stack frames

factorial (n) n frames $O (n)$ space

An iterative loop uses **one** frame $O (1)$

Deep recursion on a microcontroller
overflows the stack

The Time-Space Tradeoff

You can often **buy time with memory**

Compute the answer every time:
less memory, more time

Precompute a **table** of answers:
more memory, less time

Example: An ADC Lookup Table

A 12-bit ADC gives counts 0 .. 4095

static float PSI_TABLE[4096] ; 16 KB

Fill it once: $O(n)$ time

Then every reading: PSI_TABLE[counts] $O(1)$

No arithmetic at all in the sampling loop

Complexity Is Not Wall Clock

$O(n \log n)$ beats $O(n^2)$ - **eventually**

For $n = 20$, a simple quadratic sort often wins

Constants, cache behaviour, and memory layout
all matter in practice

Complexity tells you how it **scales**

Measuring in Practice

C: `#include <time.h>`
 `clock_gettime ( CLOCK_MONOTONIC, &t0 ) ;`

C++: `auto t0 = std::chrono::steady_clock::now ( ) ;`

Rust: `let t0 = std::time::Instant::now ( ) ;`
 `println! ( "{:?}", t0.elapsed ( ) ) ;`

Analyze First, Then Measure

Analysis tells you what to **expect**

Measurement tells you what you **got**

If they disagree, you have learned something:
a hidden cost, a surprise allocation,
or a wrong assumption about n

Costs You Should Know

Operation	Cost
Index an array: $a[i]$	$O(1)$
Linear search	$O(n)$
Binary search (sorted)	$O(\log n)$
Sort	$O(n \log n)$
Append to a vector	$O(1)$ amortized
Insert at the front	$O(n)$
Hash lookup	$O(1)$ average

Amortized Cost

`v.push_back ( x ) ;` `v.push ( x ) ;`

Usually $O(1)$ - just write and bump the length

Occasionally the buffer is **full**:

 allocate double the size, copy n elements

Averaged over many pushes: **$O(1)$ amortized**

Read the Documentation

The C++ standard and the Rust docs **state** the complexity of every operation

<code>std::sort</code>	$O(n \log n)$
<code>std::map::find</code>	$O(\log n)$
<code>Vec::push</code>	$O(1)$ amortized
<code>HashMap::get</code>	$O(1)$ average

Check before you use it in a loop

Example: Any Duplicates?

Given n readings, are any two the same?

Three ways to answer it:

1. compare **all pairs**
2. **sort**, then scan neighbours
3. use a **hash set**

Same answer. Very different cost.

Approach 1: All Pairs

```
for ( i = 0 ; i < n ; i++ )  
    for ( j = i + 1 ; j < n ; j++ )  
        if ( a[i] == a[j] ) return true ;  
return false ;
```

Time: **$O(n^2)$**

Space: **$O(1)$**

Approach 2: Sort Then Scan

```
v.sort ( ) ;  
for i in 1..v.len ( ) {  
    if v[i] == v[i-1] { return true ; }  
}
```

Time: **$O(n \log n)$** sort + **$O(n)$** scan

Space: **$O(1)$** if sorted in place

But it **reorders** the data

Approach 3: A Hash Set

C++: `std::unordered_set<int> seen ;`
 `if ( ! seen.insert ( a[i] ).second ) return true ;`

Rust: `let mut seen = HashSet::new ( ) ;`
 `if ! seen.insert ( x ) { return true ; }`

Time: **$O (n)$** average Space: **$O (n)$**

Comparing the Three

Approach	Time	Space	n = 100,000
All pairs	$O(n^2)$	$O(1)$	about 5 seconds
Sort then scan	$O(n \log n)$	$O(1)$	about 2 ms
Hash set	$O(n)$ average	$O(n)$	under 1 ms

Rough times at one billion operations per second

Common Mistakes

Calling an $O(n)$ function inside a loop
strlen, find, length of a linked list

Building a string by `+` in a loop
each join copies everything so far

Forgetting **space**: a table that does not fit

Optimizing before **measuring**

How to State It

"Binary search is $O(\log n)$ time
and $O(1)$ space."

"This loop is $O(n^2)$ because the inner
loop runs n times for each of n outer passes."

Always say **time** or **space**,
and say what **n** is

What We Can Now Do

Count the work an algorithm does

State a bound with O , Ω , or Θ

Prove a simple bound from the definition

Recognize the common classes in code

Trade space for time deliberately

Let's practice

Exercise: Using **bigopractice**